PAKISTAN INSTITUTE OF ENGINEERING AND APPLIED SCIENCES

Agentic PIEAS ERP Synchronization

A Four-Agent LangGraph System for Continuous, Unattended Data Migration into OpenEduCat

A legacy learning-management system, with no API and no webhooks, kept in perpetual synchrony with a modern Odoo-based ERP --- through bulk migration, incremental updates, and unattended deletion detection.

Specialized agents. Strict state control. Complete data integrity.

PROJECT TYPE
Internship / Systems Integration

STACK
LangGraph • Gemini • MySQL • Odoo XML-RPC

DATE
August 2026

Abstract

The PIEAS Learning Management System is a legacy application with no external API, no webhook mechanism, and no capacity to announce its own state changes. Migrating its data into a modern OpenEduCat (Odoo) Education ERP therefore requires the *target* system to actively discover changes at the source, rather than being notified of them. This report documents the design, implementation, and verification of a four-agent synchronization system built on LangGraph, in which a single **Orchestrator** agent owns all state and scheduling decisions, and three stateless workers — **Extractor**, **Transformer**, and **Loader** — execute exactly what they are instructed to, nothing more. The same pipeline drives three operational phases: an initial bulk migration, a recurring incremental synchronization driven by a strictly advancing timestamp watermark, and a slower deletion scan that detects records absent from the source via an in-memory set difference. Large-language-model reasoning (Google Gemini) is applied sparingly and cached aggressively — once per table for schema mapping, once per query shape for SQL generation — so that per-row execution remains deterministic, fast, and reproducible even across thousands of records. The system was implemented and verified end-to-end against a live MySQL source and a live Odoo 19 instance, migrating 99 records across eight related entities with zero data loss, correctly routing examination data into OpenEduCat's Examination module, and correctly distinguishing an archivable deletion from a destructive one.

Keywords — multi-agent systems, LangGraph, ERP data migration, Odoo, OpenEduCat, incremental synchronization, watermarking, large language models, XML-RPC.

Contents

1 Introduction

1.1 Problem Statement

The Pakistan Institute of Engineering and Applied Sciences (PIEAS) operates a custom-built, standalone Learning Management System (LMS) that has accumulated years of institutional data: departments, courses, batches, subjects, faculty, students, examinations, and results. The institute is migrating academic records management to **OpenEduCat**, an open-source Education ERP built on the Odoo framework, to eliminate data silos and reduce the maintenance burden of a bespoke system.

This upgrade introduces a severe technical constraint: **the source system is completely silent**. The legacy PIEAS LMS exposes no API, issues no webhooks, and has no mechanism to push a notification when a row is inserted, updated, or deleted. Any synchronization strategy must therefore treat the source purely as a passive, query-only surface — OpenEduCat must actively *hunt* for what has changed.

This asymmetry is the central design problem addressed by this report:

> *"How do you synchronize a system that cannot tell you when its data changes?"*

1.2 Design Constraints

Four constraints shaped every subsequent decision:

1. **Zero modification of the legacy source.** The PIEAS LMS schema, application, and data cannot be altered. No trigger, no dirty-flag column, no outbox table may be added to it.
2. **No notification channel.** Change detection must be pull-based, driven entirely from the target side.
3. **Correctness under partial failure.** A crashed or interrupted sync run must never silently lose a change or duplicate a record.
4. **Respect for the target's business rules.** Writes into Odoo must go through its Object-Relational Mapping (ORM) layer so that computed fields, validation, and related-record bookkeeping are honoured — not bypassed.

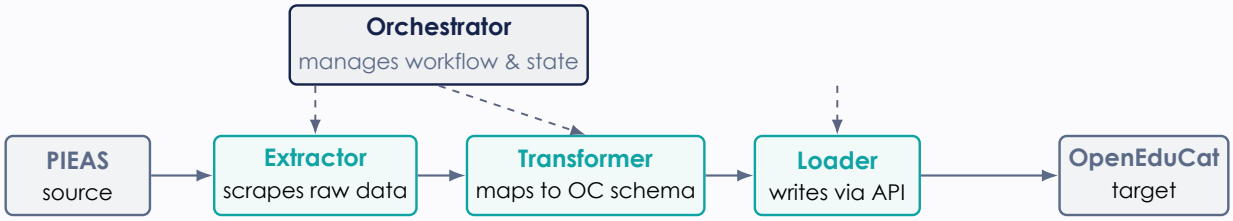
1.3 Solution Overview

The system implements a specialized four-agent architecture, orchestrated with **LangGraph**, in which exactly one agent is permitted to make decisions and hold memory, while three others are strictly stateless executors. The same pipeline structure serves three distinct operational phases — bulk migration, incremental synchronization, and deletion detection — without any change to the workers' logic, differing only in the instructions the Orchestrator issues. Large language model reasoning (Gemini) is used for schema understanding and query authorship, not for per-record decision making, keeping the system both intelligent and fast.

2 System Architecture

2.1 Design Principle: One Agent, One Job

Rather than a monolithic script, the solution is built as a four-agent ecosystem governed by a strict separation of concerns: a single agent decides, three agents execute.

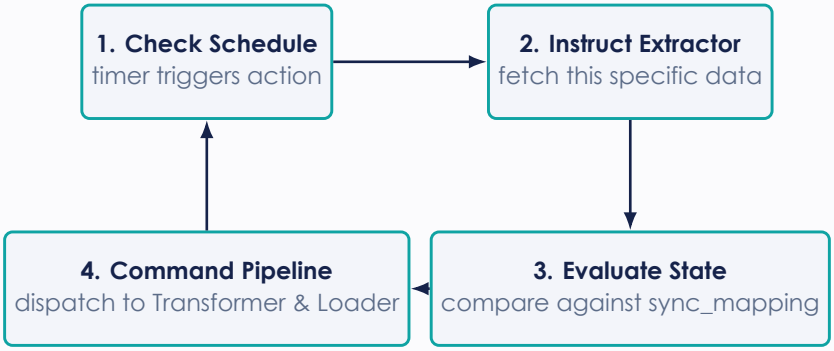


2.2 Agent Responsibilities

Agent	Holds State	Role
Orchestrator	**Yes — exclusively**	The only decision-maker in the system. Reads the state store to determine each entity's phase (bulk vs. incremental), writes natural-language extraction instructions, evaluates fetched rows against its memory of prior syncs, and is the sole agent permitted to advance the synchronization watermark.
Extractor	No	The pure fetcher, and the *only* agent that ever touches PIEAS. Takes an instruction, authors (or reuses cached) read-only SQL, and returns raw rows. Performs no comparisons and makes no decisions.
Transformer	No	A pure function. Maps the PIEAS relational schema onto OpenEduCat's Odoo models using a cached, per-entity mapping plan. Identical logic executes regardless of which phase invoked it.
Loader	No	The writer. Executes the final action against Odoo over XML-RPC: create, update, or archive. On rejection, consults the Orchestrator's diagnostic reasoning to decide whether to retry, repair, or quarantine the record.

2.3 The Orchestrator's Control Loop

Each synchronization run advances through a four-step cycle, repeated once per configured entity:



The Orchestrator's memory, the `sync_mapping` table, bridges every PIEAS primary key to its corresponding Odoo record ID, and is the single source of truth consulted at step 3.

3 Operational Phases

A single pipeline drives three phases; only the instruction issued by the Orchestrator differs.

Phase	Trigger	Extractor Instruction	Loader Operation
1. Bulk Migration	State store is empty	Fetch every row, paginating until exhausted	Predominantly `create`
2. Incremental (Fast Pulse)	Timer, every N minutes	`WHERE last_updated > watermark`	`upsert`
3. Deletion Scan (Slow Pulse)	Timer, once daily	Fetch the `id` column of every current row	`archive`

3.1 Why Two Distinct Rhythms

Because PIEAS cannot push notifications, the Orchestrator must actively hunt for two structurally different kinds of change, and a single polling strategy cannot catch both.

The Fast Pulse --- Timestamp Watermarks

Every source table carries a `last_updated` column, maintained automatically by MySQL (`ON UPDATE CURRENT_TIMESTAMP`). The Orchestrator records a `last_synced_at` watermark per entity and instructs the Extractor to fetch only rows strictly newer than it. After a successful load, the watermark advances to the newest timestamp seen — and *only* then, so a partially-failed batch is safely retried on the next tick rather than silently skipped.
Anti-pattern, deliberately avoided: a boolean `sync_pending` flag. If a row changes again while it is mid-sync, naively resetting the flag to `0` after the first sync silently drops the second change. A monotonically advancing timestamp cannot lose a write in this way.

The Slow Pulse --- Ghost Detection via Set Difference

A `WHERE last_updated > watermark` query is structurally incapable of finding a row that no longer exists — there is nothing left to query. Deletion detection is therefore a different operation entirely: the Extractor pulls *only* the current list of primary keys from PIEAS, and the Orchestrator computes the set difference against the primary keys it remembers as active in `sync_mapping`. Anything present in memory but absent from the fresh fetch is a *ghost* — conceptually equivalent to:

```
SELECT m.pieas_id FROM sync_mapping m
LEFT JOIN pieas_table s ON s.id = m.pieas_id
WHERE s.id IS NULL AND m.status = 'active'
```

This scan is heavier than the fast pulse (it must enumerate every live ID) but far cheaper than re-fetching every column, so it is scheduled far less frequently.

3.2 Archiving, Not Destroying

When the slow pulse confirms a deletion, the Loader does not delete the corresponding Odoo record. It sets `active = False` wherever the target model supports it.

Why Archive

- **Historical integrity.** Deleting a student record would cascade into — or orphan — their grade history and past enrolments. An archived record preserves the full academic trail.
- **Reversibility.** A PIEAS-side rename or a temporary status toggle can look identical to a deletion from the outside. Archiving is safely undone: if the record reappears on a later fetch, the next incremental sync simply un-archives it.
- **Model-aware fallback.** Not every Odoo model exposes an `active` field — line-item models such as `op.exam.attendees` have none. The Loader inspects the live schema at runtime and falls back to a true removal only when no archived state exists to move the record into.

4 Large-Language-Model Strategy

4.1 Reasoning Once, Executing Deterministically

A naive design might invoke an LLM for every single record synchronized. This was deliberately rejected: it is slow, expensive, non-deterministic across runs, and unnecessary. Gemini is instead used to reason about *shape* — schemas, mappings, and query structure — exactly once per distinct need, with the result cached to disk. Per-row execution then runs as ordinary, deterministic Python driven by that cached plan.

Agent	What Gemini Is Asked	Frequency
Orchestrator	Plan each entity's phase and author the Extractor's natural-language instruction	Once per run
Extractor	Translate the instruction into a single, validated `SELECT` statement	Once per (entity, phase) — *cached*
Transformer	Map MySQL columns onto Odoo's live `fields_get()` response	Once per entity — *cached*
Loader	Triage a rejected write and choose a recovery action	Once per distinct error signature

As a direct consequence, a bulk migration of thousands of rows costs on the order of a dozen model calls, not thousands, and produces identical output on every re-run — an essential property for a system that must be demonstrably reliable, not merely occasionally correct.

4.2 The Model Proposes; the Live Schema Decides

Every artifact an LLM produces is treated as an untrusted proposal and passed through a deterministic sanitizer before it is used:

- Generated SQL must parse as exactly one `SELECT` statement, reference only a whitelisted table, and use exactly the expected number of bind parameters — otherwise it is rejected outright and a hand-written default query is used instead.
- Mapping plans are filtered against Odoo's live `fields_get()` response. Any field the model names that is not writable — notably **readonly, related-and-stored** fields such as `op.exam.course_id`, which Odoo derives automatically from `session_id` — is silently dropped before it can cause a write failure.
- Foreign keys the model overlooks are recovered independently from MySQL's `information_schema`, so relational correctness never depends solely on the model noticing a key.
- If Gemini is unreachable or its daily free-tier quota is exhausted, every agent falls back to a deterministic plan, and the synchronization run completes rather than halting.

5 Implementation

5.1 Technology Stack

Layer	Choice
Orchestration	LangGraph `StateGraph`, four nodes, one conditional router
LLM	Google Gemini (`gemini-3.1-flash-lite`), via `langchain-google-genai`
Source	MySQL 8.0, read-only access
Target	OpenEduCat 19.0 on Odoo 19, written over **XML-RPC**
State store	Local SQLite (`sync_mapping`, `sync_watermark`, `sync_run`)
Scheduler	APScheduler, cadence declared as plain constants

5.2 Why XML-RPC, Not a Direct Database Write

Approach	Verdict
Direct PostgreSQL write	**Rejected** — fast, but skips computed fields, validation, and related-record bookkeeping; risks half-formed records that fail silently later.
REST API	Alternative — clean and ORM-safe, but availability depends heavily on the specific hosting setup and Odoo edition.
XML-RPC	**Selected** — ships with every self-hosted Odoo instance, enforces every business rule through the ORM, and is comprehensively documented.

5.3 Entity Routing

Declared once, in dependency order, so that a child's foreign keys always resolve to real Odoo IDs by the time the child itself is synchronized:

MySQL Table	OpenEduCat Model	Module
`departments`	`op.department`	Core
`courses`	`op.course`	Core
`batches`	`op.batch`	Core
`subjects`	`op.subject`	Core
`faculty`	`op.faculty`	Core
`students`	`op.student`	Core
`exams`	`op.exam`	**Examination**
`exam_results`	`op.exam.attendees`	**Examination**

Adding a ninth entity requires a single line in configuration; the agents discover its columns, foreign keys, and target fields at run time rather than through hardcoded schema knowledge.

5.4 The State Store

The Orchestrator's memory is the only place synchronization bookkeeping lives — deliberately kept out of both PIEAS (which must not be modified) and Odoo (which should not accumulate sync-specific clutter).

Listing 1: Core schema of the Orchestrator-owned state store

```
sync_mapping(entity, pieas_id, odoo_model, odoo_id,
             row_hash, status, last_synced_at)
  PRIMARY KEY (entity, pieas_id)
  -- the id bridge, and the index the deletion scan diffs against

sync_watermark(entity, watermark, last_deletion_scan)
  -- the strictly-advancing high-water mark per entity
```

A per-row content hash is also stored, so that a row whose `last_updated` timestamp changed but whose meaningful content did not (a re-save with no real edits, for instance) is skipped rather than triggering a redundant write.

6 Engineering Challenges and Resolutions

Building an agentic system that writes to production-shaped software surfaced real failure modes that a purely theoretical design would not anticipate. Three are documented here because each reveals a general principle, not just a one-off bug.

Challenge 1 --- An LLM Hallucinated a Foreign Key

During the first bulk migration, the Transformer's Gemini-authored mapping plan for both `students` and `faculty` included a hardcoded default: `partner_id: 1`. Because `op.student` and `op.faculty` both use Odoo's `_inherits` mechanism against `res.partner`, every one of the twenty students was bound to the *same* underlying partner record, and each subsequent write silently overwrote the previous student's name — corrupting the company record itself in the process.

Resolution The sanitizer that filters every LLM-produced mapping plan now rejects *any* literal, hardcoded value proposed for a relational field (`many2one`, `one2many`, `many2many`), unconditionally. A relation may only ever be populated by resolving a source-side foreign key through `sync_mapping`. This closes the entire class of failure, not merely this one instance of it.

Challenge 2 --- A Failed Call Silently Cached a Broken Plan

An early version of the LLM helper caught every exception and returned an empty fallback plan on failure. When a transient error occurred mid-run, that empty plan was written to the on-disk cache and reused on every subsequent run — producing records that were missing every required field, indefinitely, until someone thought to delete the cache by hand.

Resolution Failure now raises explicitly rather than degrading into a plausible-looking empty result. Each caller decides for itself whether a deterministic fallback is safe (it is, for SQL generation and phase planning) or unsafe (it is not, for schema mapping, where no hand-written default can guess that `reg_no` means `gr_no`). Critically, a plan is only persisted to the cache *after* passing validation — a bad run can no longer poison future runs.

Challenge 3 --- The Free-Tier Quota Is Per Model, Per Day

The first two full bulk-migration attempts exhausted Gemini's free-tier quota, which is capped at twenty requests per day *per model*. Because the client library's default retry behaviour used long exponential backoff, failed calls appeared to hang for over two minutes rather than failing fast.

Resolution The system moved to `gemini-3.1-flash-lite`, whose daily allowance comfortably covers a full run; quota exhaustion is now detected explicitly (HTTP 429, parsed for the server's own suggested retry delay) rather than masked by generic retry logic; and because the per-entity mapping and SQL plans are cached after the first successful run, a *warm* re-run costs a single Gemini call regardless of table size.

7 Verification and Results

The system was verified end-to-end against a live MySQL instance and a live Odoo 19 / OpenEduCat installation — not mocked.

Test	Method	Result
Bulk migration	Seed 99 rows across 8 tables; run full migration	**99/99, 0 failures**
Update propagation	Edit a student's name and phone in MySQL; run incremental sync	**Reflected correctly**
Examination routing	Insert a new exam row; confirm it lands in `op.exam`	**Correct module**
Deletion handling	Delete a student in MySQL; run the deletion scan	**Archived, not destroyed**
Watermark safety	Run incremental sync twice with no source changes	**Zero rows on 2nd run**
Unattended scheduling	Run the scheduler; edit MySQL live; observe the next tick	**Caught unattended**

7.1 Representative Verification Evidence

After the deletion test, the archived record was confirmed to be fully intact and simply hidden from default views:

Listing 2: A student archived, not destroyed, after her row vanished from PIEAS

```
{'id': 97, 'name': 'Rabia Sultan', 'gr_no': 'PIEAS-23-CHE-001',
 'active': False, 'birth_date': '2005-07-07',
 'email': 'rabia.sultan@student.pieas.edu.pk'}

Her exam results: 2 rows, still present and unmodified.
```

The final reconciled state matched the source exactly: nineteen active and one archived student against MySQL's remaining nineteen rows; eleven examinations against ten original plus one inserted during testing.

8 Conclusion and Future Work

8.1 Summary

This project demonstrates that a small, strictly-scoped team of specialized agents — one decision-maker and three disciplined executors — can solve a synchronization problem that a monolithic script would handle only clumsily. The core insight is architectural: separating *who decides* from *who executes* makes the system's behaviour auditable, its failure modes narrow, and its LLM usage cheap, because reasoning happens only where genuine ambiguity exists (schema mapping, query authorship) and never in the hot path of per-record execution.

Equally important is what the engineering challenges in Section 6 demonstrate: an LLM embedded in a data pipeline must be treated as an untrusted proposer whose output is always validated against the live, authoritative schema before it can act — never as an oracle to be trusted directly. Every mapping plan, every generated query, and every recovery decision passes through a deterministic sanitizer before it is permitted to touch production data.

8.2 Future Work

- **Parallel entity processing.** Independent entities without a foreign-key relationship (e.g. `departments` and unrelated lookup tables) could sync concurrently rather than sequentially.
- **Conflict detection.** The current design assumes PIEAS is the sole source of truth; a future iteration could detect and flag the case where an OpenEduCat record has been manually edited since the last sync, rather than silently overwriting it.
- **Observability dashboard.** The `sync_run` audit log already captures every batch's outcome; a lightweight web view over it would give administrators visibility without requiring direct SQLite access.
- **Additional entity coverage.** Attendance, fees, and library records follow the same routing pattern and could be added with no change to the core agents.

Specialized agents. Strict state control. Complete data integrity.